Module-VIII
Lecture-I, II and III
Fault Simulation

1. Introduction

In the last lecture we learnt how to determine a minimal set of faults for stuck-at fault model in a circuit. Following that a test pattern is to be generated for each fault which can determine the presence/absence of the fault under question. The procedure to generate a test pattern for a given a fault is called Test Pattern Generation (TPG). Generally TPG procedure is fully automated and called Automatic TPG (ATPG). Let us revisit one circuit discussed in last lecture and briefly note the steps involved in generating a test pattern for a fault. Consider the circuit in Figure 1 and s-a-1 fault at the output of gate G1. Three steps are involved to generate a test pattern

a. Fault Sensitization: As the output net of G1 is stuck-at-1, we need to drive it to 0 to verify the presence/absence of the fault. So output of G1 is marked 0.
b. Fault Propagation: Affect of the fault is to be propagated to a primary output (Output of G6, in this example). In this case, the propagation path is ``output of G1 to output of G6". For enabling this fault to propagate by this path, other inputs to G6 (from gate G2, G3, G4 and G5) are to be made 1. Affect of fault at the primary output is "1 if fault is present and 0 if fault is not present".
c. Justification: Determination of values at primary inputs so that Fault sensitization and Fault propagation are successful. To sensitize fault (0 at output of G1), I1=0 and I2 through I5 are 1. To make outputs of gates G2, G3, G4 and G5 1, primary inputs I6 through I25 are made 1.

So test pattern is I1=0 and I2 through I25 are 1. It may be noted that there are 2^5-1 choices to sensitize fault (apply 0 at output of G1); all input combinations of G1 except I1=1, I2=1, I3=1, I4=1 and I5=1. However, to propagate the fault there is only one pattern (primary inputs I6 through I25 are made 1). TPG procedure would generate any one of the patterns given in Table 1

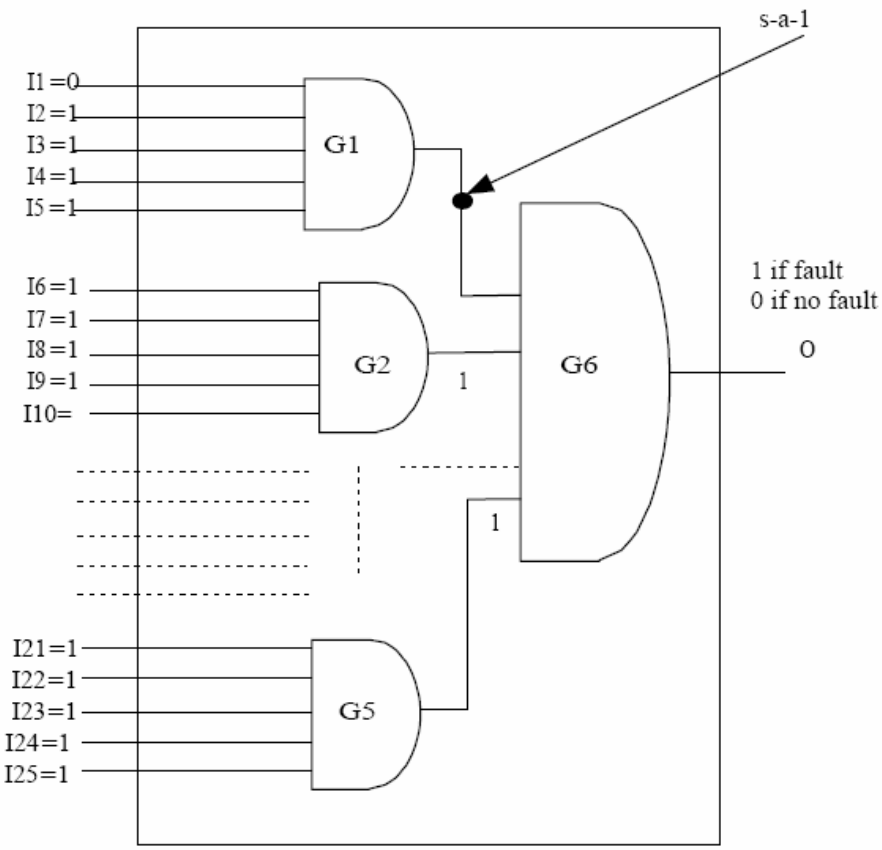


Figure 1. s-a-1 fault in net OG1 with input test pattern

Table 1. Test patterns for the s-a-1 fault in the circuit of Figure 1

Test Pattern No.	Test Pattern I1 I2 I3 I4 I5 I6……I25	Output
1	0 0 0 0 0 11111111111111111111	1 if fault 0 if no Fault
2	0 0 0 0 1 11111111111111111111	1 if fault 0 if no Fault
……	……..	……..
2^{25}	1 1 1 1 0 11111111111111111111	1 if fault 0 if no Fault

Now the question is, do we require these three steps for all faults? If this is the case, then TPG would take significant amount of time. However, we know that one test pattern can test multiple faults. In this lecture we will see how TPG time can be reduced using the concept of "one TP can test multiple faults".

Let us understand the basic idea of TPG time reduction on the example of Figure 1. Let us not try to generate a test pattern for a fault, rather apply a random pattern as see what faults are covered. Table 2 shows two random patterns and the faults they test. For example, the random pattern No. 1 generates a 0 at the output of G1, 1 at the outputs of gates G2 through G5 and 0 at the output of G6. It can be easily verified that this pattern can test s-a-1 fault at two locations.

- output of G1
- output of G6

In case of faults, output at O is 1 and under normal conditions it is 0.

However, the next random pattern (No.2) can detect a lot more s-a faults in the circuit. It can test s-a-0 faults in all nets of the circuit. For example, let us take s-a-0 fault at output of G3. It can be easily verified that if this pattern is applied, O will be 0 if fault is present and 1 other wise. Similarly, we can verify that all possible s-a-1 faults in the circuit (31 in number) would be tested.

Table 2. Random patterns and the faults they test in the circuit of Figure 1

Pattern No.	Random Pattern I1 I2 I3 I4 I5 I6……I25	Faults Detected
1	1 0 0 0 1 11111111111111111111	s-a-1 at net "output of G1" s-a-1 at net "output of G6"
2	1 1 1 1 1 11111111111111111111	s-a-0 faults in all the nets of the circuit

So we can see that by using 2 random patterns we have generated test patterns for 33 faults. On the other hand if we would have gone by the "sensitize-propagate-justify" approach these three steps would have been repeated 33 times. Now let us enumerate the steps used to generate the random test patters and determine the list of faults detected (*called faults covered*).

1. Generate a random pattern
2. Determine the output of the circuit for that random pattern as input
3. Take fault from the fault list and modify the Boolean functionally of the gate whose input has the fault. For example, in the circuit in Figure 1, the s-a-1 fault at the output of gate G1 modifies the Boolean functionality of gate G6 as 1 AND I2 AND I3 AND I4 AND I5 (which is equivalent to I2 AND I3 AND I4 AND I5) .
4. Determine output of the circuit with fault for that random pattern as input.
5. If the output of normal circuit varies from the one with fault, then the random pattern detects the fault under consideration.
6. If the fault is detected, it is removed from the fault list.
7. Steps 3 to 6 are repeated for another fault in the list. This continues till all faults are considered.
8. Steps 1 to 7 are repeated for another random pattern. This continues till all faults are detected.

In simple words, new random patterns are generated and all faults detected by the pattern are dropped. This continues till all faults are detected by some (random) pattern. Now the question arises, "How many random patterns are required for a typical circuit"? The answer is very high. This situation is identical to the game of "Balloon blasting with air gun". If there are a large number of balloons in the front, aim is not required and almost all shots (even without aim) will make a successful hit. However, as balloons become sparse blind hits would rarely be successful. So, the balloons left after blind shooting are to be aimed and shot. The same is true for testing. For initial few random patterns faults covered will be very high. However, as the number of random patterns increase the number of new faults being covered decreases sharply; this phenomenon can be seen from the graph of Figure 2. It may be noted that typically beyond 90% fault coverage, it is difficult to find a random pattern that can test a new fault. So, for the remaining 10% of faults it is better to use the "sensitize-propagate-justify" approach. These remaining 10% of faults are called *difficult to test faults.*

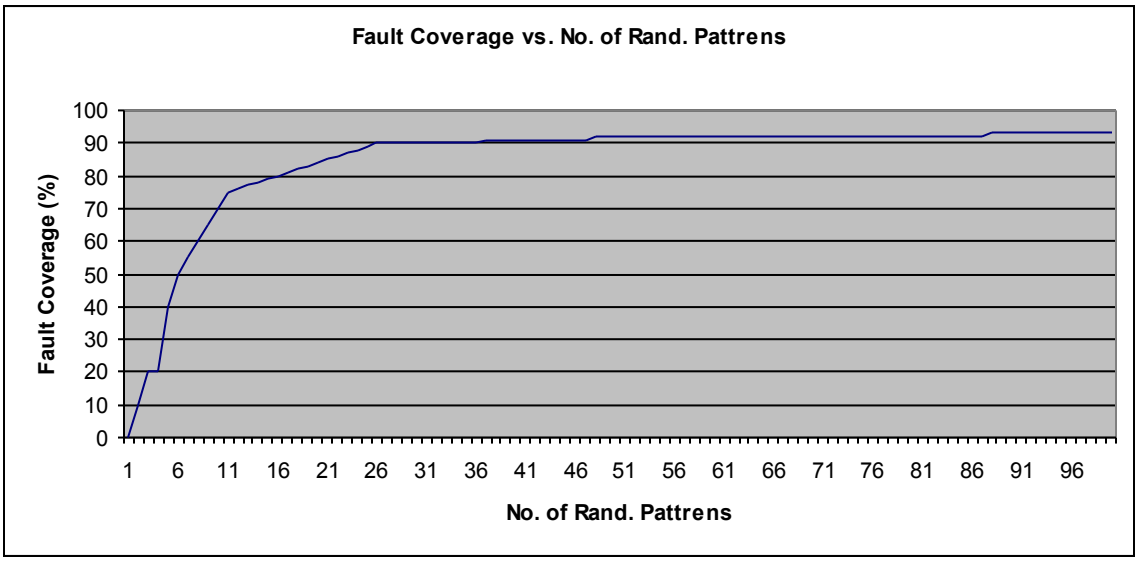


Figure 2. Typical scenario for fault coverage versus random patterns

From the discussion we conclude that TPG can be done in two phases

1. Use random patters, till a newly added pattern detects a reasonable number of new faults
2. For the remaining faults, apply "sensitize-propagate-justify" approach

Also, it may be noted that determining the output of the circuit for a given input (with and without fault) is the key step for "random pattern based TPG". In this lecture we will first algorithms for computing the output of a circuit given an input; this is called *circuit simulation*. Following that we will discuss efficient techniques to determine faults covered by random patterns, called *fault simulation*.

2. Circuit Simulation

What is circuit simulation?

The imitative representation of the functioning of circuits by means of another alternative, a computer program say, is called simulation. In digital laboratory (of undergraduate courses), "breadboards" were used to design systems using discrete chips. The same system can be developed using a single VLSI chip. However, we still prefer to do the breadboard design first, as the proof of concept. Breadboard based designs are used as proof of concept because we can predict the result of building the VLSI chip without actually building it. Further, errors can be easily rectified in the breadboard. So we may say that breadboard is a "hardware based" simulation of the VLSI chip. Breadboard has been used over decades for pre-design validation, however, is too cumbersome for large designs. Today, breadboards have been replaced by computer simulations. A circuit is converted into a computer program which generates the output given the input. Obviously, changes due to errors discovered are simple (than breadboard) as one needs to change only a computer program. There are several schemes of circuit simulation [1,2,3], and here we will see the ones related to fault simulation for the stuck-at fault model.

Compiled Code Simulation

As the name suggests, Compiled Code method of simulation involves describing the circuit in a language that can be compiled and executed on a computer. The circuit description can be in a Hardware Description Language such as VHDL or Verilog [4] or simply described in C. Inputs, outputs and intermediary nets are treated as variables in the code which can be Boolean, integer, etc. Gates such as AND, OR, etc., are directly converted into program statements (using bit wise operator). For every input pattern, the code is repeatedly executed. Figure 3(A) shows a simple digital circuit and the corresponding C code required for its simulation is given in Figure 3(B). It may be noted that all 4 inputs, two intermediary nets (OG1 and OG2) and Output are represented as integer variables. All the gates are converted into single statements using bit wise operators.

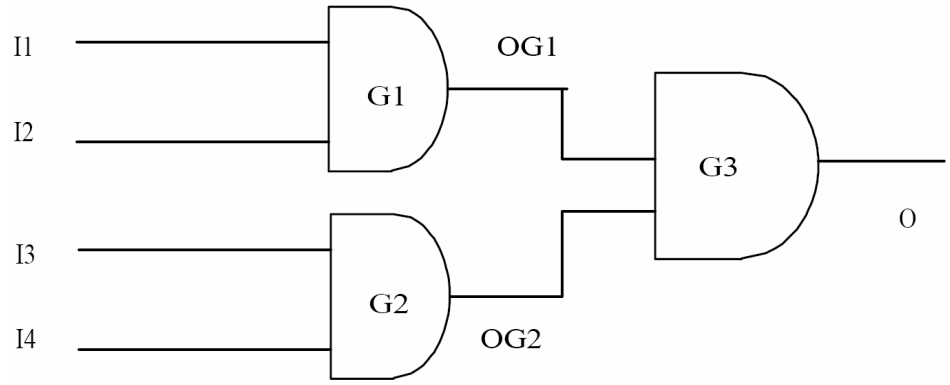


Figure 3(A). A Digital Circuit

```
# include<stdio.h>

main()
{
 int I1,I2,I3,I4,OG1,OG2,O;
 printf("Input the Values of I1, I2, I3 and I4”);
scanf("%d", &I1);
scanf("%d", &I2);
scanf("%d", &I3);
scanf("%d", &I4);
  OG1 = I1 & I2;
 OG2 = I3 & I4;
O = OG1 & OG2;
 printf("\n Output of Circuit is %d",O);
}
```

Figure 3(B). C code for the circuit of Figure 3(A) required for compiled code simulation

Compiled code simulation is one of the simplest techniques of all circuit simulators. However, for every change in input the complete code is executed. Generally, in digital circuits, only 1-10% of signals are found to change at any time. For example, let us consider the circuit given in Figure 3(A) where input I1 changes from 1 to 0; this is

illustrated in Figure 4. We can easily see that change in input modifies only 3 out of 8 lines. On the other hand a compiled code version would require reevaluation of all the 8 variables. Next we will discuss event-driven simulators which may not require evaluating all gates on changing of an input.

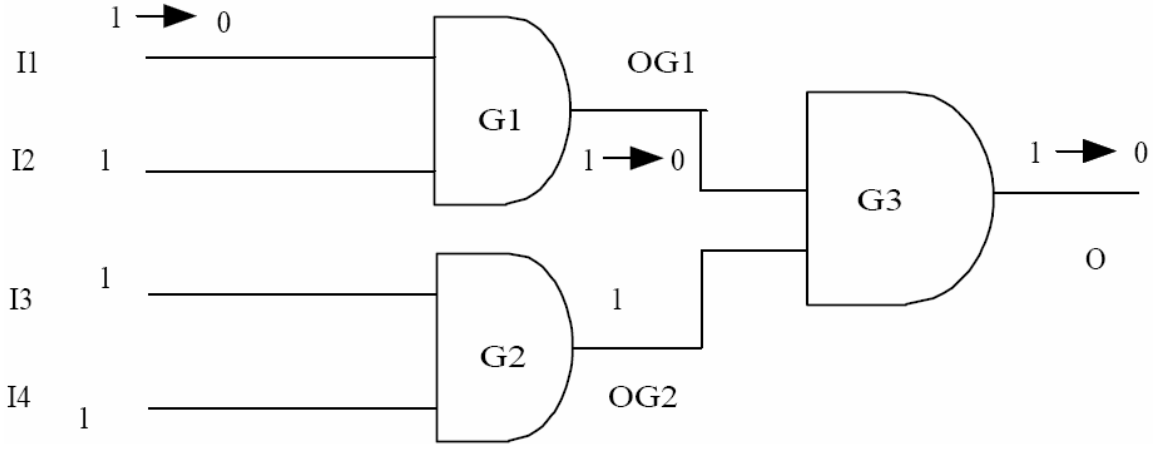
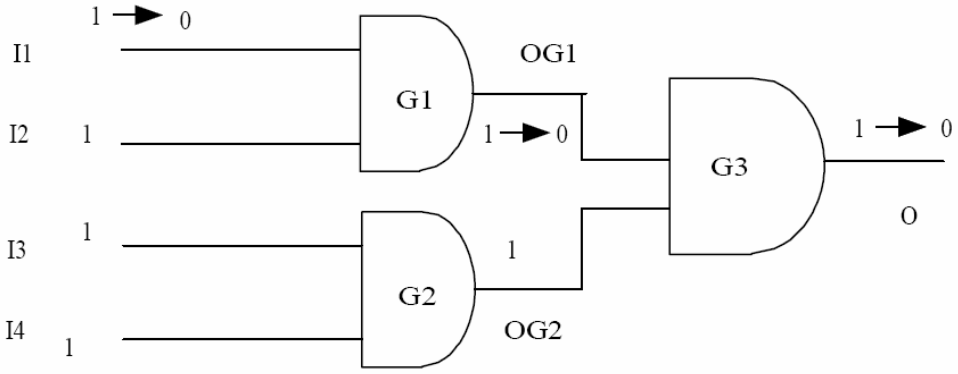


Figure 4. Changes in signal values in a circuit for change in input

Event Driven Simulation

Event-driven simulation is a very effective scheme for circuit simulation as it is based on detection of any signal change (event) to trigger other signal(s). So, an event triggers new events, which in turn may trigger more events; the first trigger is a change in primary input. Let us consider a circuit (the same one as of Figure 4) illustrated in Figure 5. Suppose, all signals are in steady-state by inputs I1=1, I2=1, I3=1,I4=1 when a new pattern I1=0, I2=1, I3=1,I4=1 is applied to primary inputs; in other words I1 changes from 1 to 0. Change in I1 is the first event. Outputs of gates, whose inputs are dependent on events which have changed, become active and are placed in an activity list; I1 drives gate G1 and OG1 becomes a part of activity list. In the next step (time instant) each gate from the activity list is evaluated to determine whether its output changes; if output changes it becomes a triggering event and all gates driven by this output becomes a part of activity list. In this example, output of G1 changes (from 1 to 0) and this event adds O (output of G3) to activity list. On the other hand, if the output (of the gate under consideration) remains same then it does not correspond to triggering event and it does not generate any new gates for activity list. Here, output of G2 does not change and so it

cannot add any gate to activity list. Once a gate from activity list is evaluated it is removed. The process of evaluation stops when the activity list becomes empty. In this example, in step 3, O changes from 1 to 0 and this makes the activity list empty. So, in event driven simulation we performed only three evaluations which are the number of signal changes in the circuit. An event-driven simulator only does the necessary amount of work. For logic circuits, in which typically very few signals change at a time, this can result in significant savings of computing effort.



Time	Scheduled Events	Activity List
t=0	I1 =0	OG1
t=1	OG1=0	O
t=2	O=0	

Figure 5. Example of event driven simulation

3. Algorithms for Fault Simulation

A fault simulator is like an ordinary simulator, but needs to simulate two versions of a circuit (i) without any fault for a given input pattern and (ii) with a fault inserted in the circuit and for the same input pattern. If the outputs under normal and faulty situation differ, the pattern detects the fault. Step (ii) is repeated for all faults. Once a fault is detected it is dropped and is not considered further during fault simulation by next random pattern. The basic idea is illustrated in Figure 6.

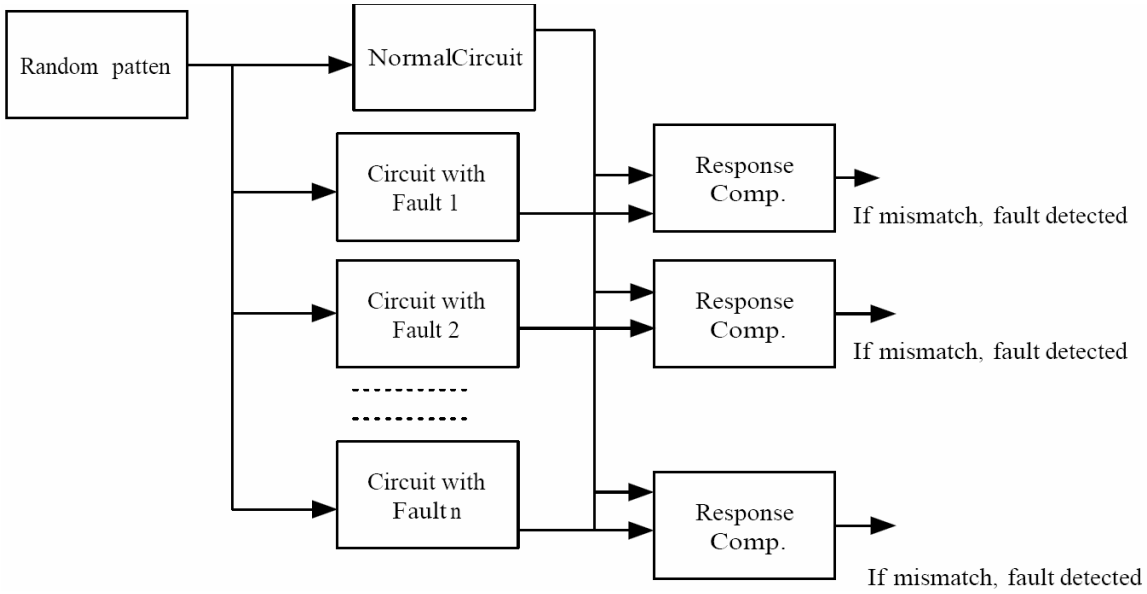


Figure 6. Basic principal of fault simulation

The procedure is simple, but is too complex in terms of time required. Broadly speaking, time required is $\sum_{i=i}^{no\ of\ random\ patterns} faults\ for\ i^{th}\ random\ pattren \times simulation\ time$.

Now we discuss algorithms which attempt to reduce this time. Basically such algorithms are based on two factors

1. Determine more than one fault that is detected by a random pattern during one simulation run
2. Minimal computations when the input pattern changes; the motivation is similar to event driven simulation over complied code simulation.

3.1 Serial Fault Simulation

This is the simplest fault simulation algorithm. The circuit is first simulated (using event driven simulator) without any fault for a random pattern and primary output values are saved in a file. Next, faults are introduced one by one in the circuit and are simulated for the same input pattern. This is done by modifying the circuit description for a target fault and then using event driven simulator. As the simulation proceeds, the output values (at different primary outputs) of the faulty circuit are dynamically compared with the saved true responses. The simulation of a faulty circuit halts when output value at any primary output differs for the corresponding normal circuit response. All faults detected are dropped and the procedure repeats for a new random pattern. This procedure is illustrated in Figure 7. The circuit has two inputs and two primary outputs. For the random input pattern I1=1, I2=1 the output is O1=0, O2=1 under normal condition. Now let us consider a s-a-0 fault in I2. Event driven simulation (with scheduled events and activity list) for the circuit for input pattern I1=1, I2=1 and s-a-0 fault at I2 is shown in Table 3. It may be noted that that event driven simulation for this circuit requires 4 steps (t=0 to t=3). However, at step t=2, we may note that value of O2 is determined as 0, but under normal condition of the circuit O2 is 1. So, the s-a-0 fault in I2 can be detected by input pattern I1=1, I2=1 when O2 is 0. In other words, for input pattern I1=1, I2=1 the s-a-0 fault in I2 is manifested in primary output O2. As the fault is manifested in at least one primary output line, we need not evaluate other outputs (for that input pattern and the fault). In this example, for s-a-0 fault at I2 (after t=3 step of simulation), input pattern I1=1, I2=1 gives O1=0, O2=0; so fault can be detected ONLY at O2. However, we need not do this computation----"if only one primary output line differs for an input pattern under normal and fault condition that input pattern can test the fault".

So, s-a-0 fault at I2 is dropped (i.e., determined to be tested by pattern I1=1, I2=1) after t=2 steps of the event driven simulation of the faulty circuit. To summarize, detection of stuck at faults in a circuit using event driven fault simulation may save computation time, as for many cases all the steps need not be carried out.

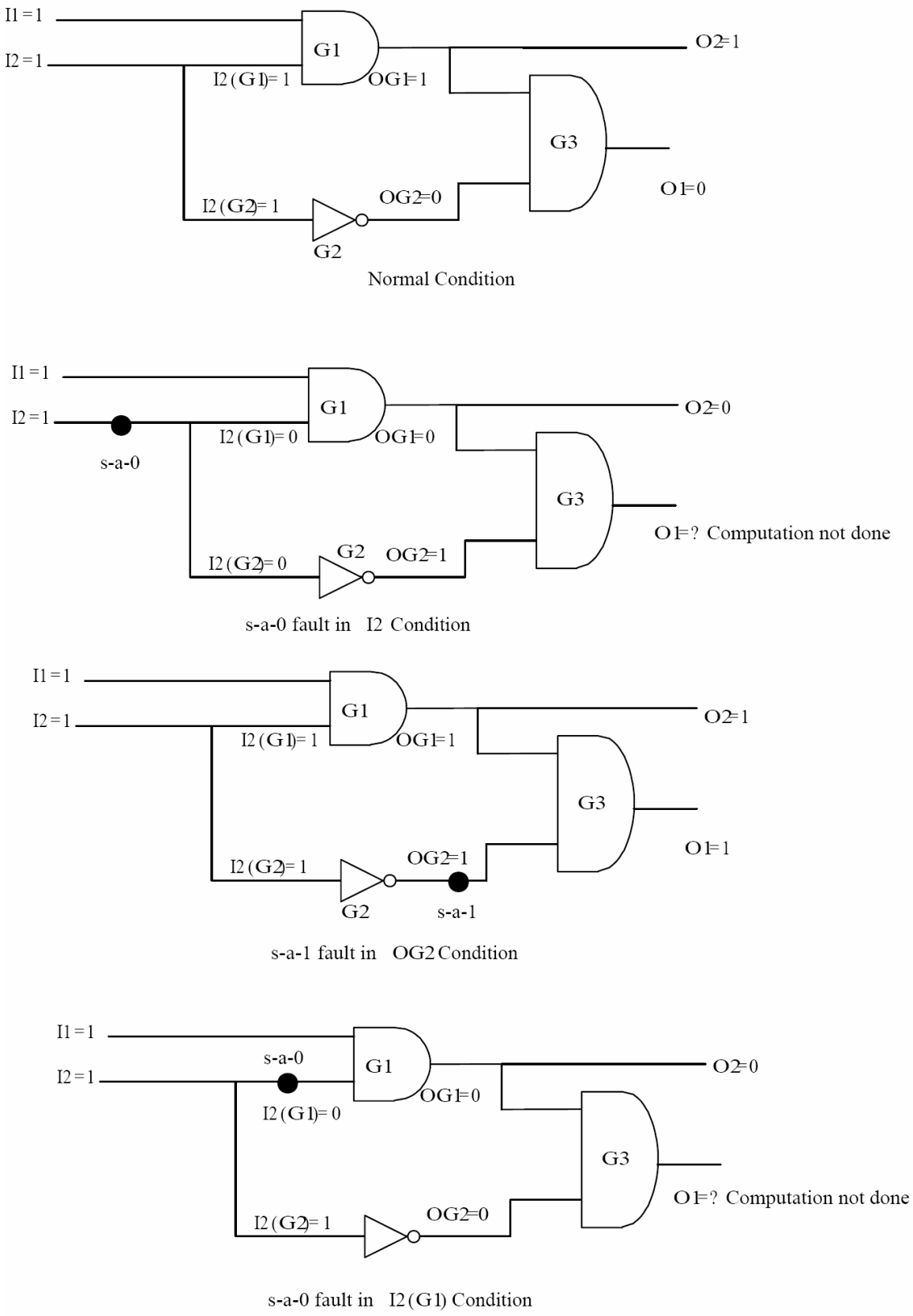


Figure 7. Illustration serial fault simulation

Table 3. Event driven simulation for circuit of Figure 7 for input pattern I1=1, I2=1 and s-a-0 fault in I2

Time	Scheduled Event	Activity List
t=0	I1=1, I2=1	I2(G1), OG1,I2(G2),O2
t=1	I2(G1)=0,I2(G2)=0	OG1, OG2,O2
t=2	OG1=0**,O2=0**,OG2=1	O1
t=3	Not Required as O2=0 in faulty situation while O2=1 in normal condition	

Now we discuss the cases for the other faults shown in Figure 7.

Event driven simulation for the circuit (of Figure 7) for input pattern I1=1, I2=1 and fault s-a-1 in OG2 is shown in Table 4. It may be noted that the fault s-a-1 in OG2 is detected by input pattern I1=1, I2=1 at output O1; in case of fault O1=1, were as in case of normal circuit O1=0. Also, all four steps of the event driven simulation are required to detect the fault. So, detecting s-a faults using event driven fault simulation may not always save computation time, as in the worst case all the steps may be needed.

Table 4. Event driven simulation for circuit of Figure 7 for input pattern I1=1, I2=1 and s-a-1fault in OG2

Time	Scheduled Event	Activity List
t=0	I1=1, I2=1	I2(G1), OG1,I2(G2),O2
t=1	I2(G1)=1,I2(G2)=1	OG1, OG2,O2
t=2	OG1=1**,**O2=1,OG2=1	O1
t=3	**O1=1**	

Event driven simulation for the circuit (of Figure 7) for input pattern I1=1, I2=1 and fault s-a-0 in I2(G1) is shown in Table 5. Like s-a-0 fault in I2, s-a-0 fault in I2(G1) is detected by I1=1, I2=1 at O2 and the fault simulator requires up to t=2 steps.

Table 5. Event driven simulation for circuit of Figure 7 for input pattern I1=1, I2=1 and fault s-a-0 fault in I2(G1)

Time	Scheduled Event	Activity List
t=0	I1=1, I2=1	I2(G1), OG1,I2(G2),O2
t=1	I2(G1)=0,I2(G2)=1	OG1, OG2,O2
t=2	OG1=0**,O2=0**,OG2=0	O1
t=3	Not Required as O2=0 in faulty situation while O2=1 in normal condition	

Serial fault simulation is simple, however as discussed earlier, for *n* faults the computing time is O($n \times no.\ of\ random\ pattrens \times simulation\ time$). With fault-dropping, this time can be significantly lower, especially if many faults are detected (and dropped) by random patterns used earlier. Next we will see advanced algorithms to reduce the complexity of fault simulation, mainly using two points

- Determine in one simulation, more than one fault if they can be detected by a given random pattern
- Use information generated during simulation of one random pattern for the next set of patterns.

Before we go for the advanced algorithms, a small question remains. When we are talking of event driven simulation, there is no question for nets being stuck at 0 or 1. So how can we use standard event driven simulator to simulate circuits having stuck at faults. Let us look at Table 3, t=1. We may note that I2(G1)=0, I2(G2)=0, because of the s-a-0 fault in I2; under normal condition I2(G1)=1, I2(G2)=1. It is very obvious to find how the values are obtained in faulty case. However, we need to see how we can use event driven simulator to simulate circuits with stuck at faults. It is to be noted that we cannot modify the simulator algorithm to handle faults; instead we will modify the

circuit. Modifying the circuit for simulating stuck at faults is extremely simple. If a net I say, has a s-a-0 fault, we insert a 2-input AND gate with one input of the gate fixed to 0 and the other input of the gate is driven by I. The gate(s) previously driven by I is now driven by output of the AND gate added. Similarly, for s-a-1 fault a 2-input OR gate with one input of the gate fixed to 1 is added.

Figure 8 illustrates insertion of the stuck at faults in the circuit shown in Figure 7. s-a-0 fault at I2 is inserted as follows. A 2-input AND gate with one input connected to 0 and the other to I2 is added. Now the newly added AND gate drives G1, which was earlier driven by I2.

Similarly, a 2-input OR gate with one input connected to 1 and the other to OG2, inserts s-a-1 fault at OG2. Now the newly added OR gate drives G3, which was earlier driven by OG2.

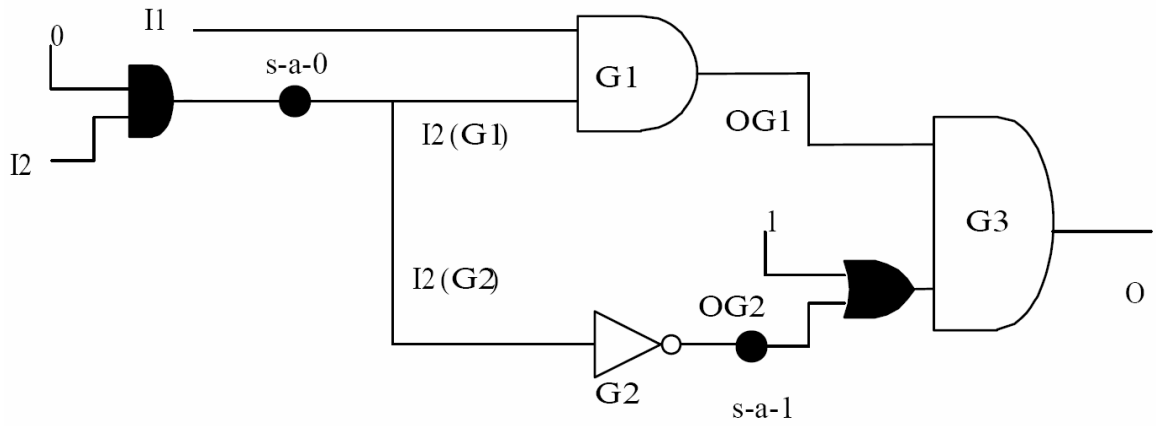


Figure 8. Insertion of faults in the circuit of Figure 7 for fault simulation in event driven simulator

To summarize, with simple modifications of the circuit, an event driven simulator can determine the output of a circuit with s-a-0 and s-a-1 faults. Henceforth, in our lectures we will not modify the circuits by inserting gates for fault simulation of stuck at faults; we will only mark the faults (as before by circles) and assume that the circuit is modified appropriately.

3.2 Parallel Fault Simulation

As discussed in the last section, serial fault simulation processes one fault in one iteration. Parallel fault simulation, as the name suggests can processes more than one fault in one pass of the circuit simulation. Parallel fault simulation uses bit-parallelism of a computer. For example, in a 16-bit computer (where a word comprises 16 bits) a logical operation (AND, OR etc.) involving two words performs parallel operations on all respective pairs of 16-bits. This allows a parallel simulation of 16 circuits with same structure (gates and connectivity), but different signal values.

In a parallel fault simulator, each net of the circuit is assigned a 1-dimensional array of width *w*, where *w* is the word size of the computer where simulation is being performed. Each bit of the array for a net *I* say, corresponds to signal at *I* for a condition (normal or fault at some point) in the circuit. Generally, the first bit is for normal condition and the other bits correspond to *w – 1* stuck at faults at various locations of the circuit.

In case of parallel simulation, input lines for any gate comprise binary words of length *w* (instead of single bits) and output is also a binary word of length *w*. The output word is determined by simple logical operation (corresponding to the gate) on the individual bits of the input words. Figure 9 explains this concept by example of an AND gate and an OR gate where *w* is taken to be 3. In the example, in gate G1, input word at I1 is 110 and that in I2 is 010. Output word 010 is obtained by bit wise ANDing of the input words at I1 and I2; this is similar to simulating the gate for three input patterns at a time namely, (i) I1=1, I2=0 (ii) I1=1, I2=1 and (iii) I1=0, I2=0.

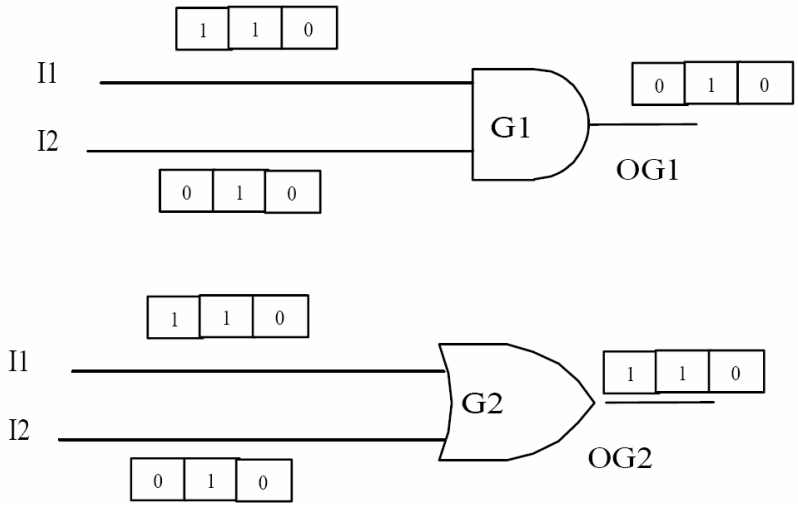


Figure 9. Example of parallel simulation

Now we will discuss how parallel fault simulation can determine status of *w-1* faults being covered or not given a random test pattern. Let us consider the same circuit used in Figure 7 for explanation of serial fault simulation. We consider the same three faults namely, (i) s-a-0 at I2, (ii) s-a-0 at I2(G1), (iii) s-a-1 at OG2 and the random pattern I1=1, I2=1. It will be shown how parallel fault simulation can discover coverage for these three faults for the input pattern I1=1, I2=1 in one iteration. As there are three faults we need *w=4*. Figure 10 illustrates parallel fault simulation for this example.

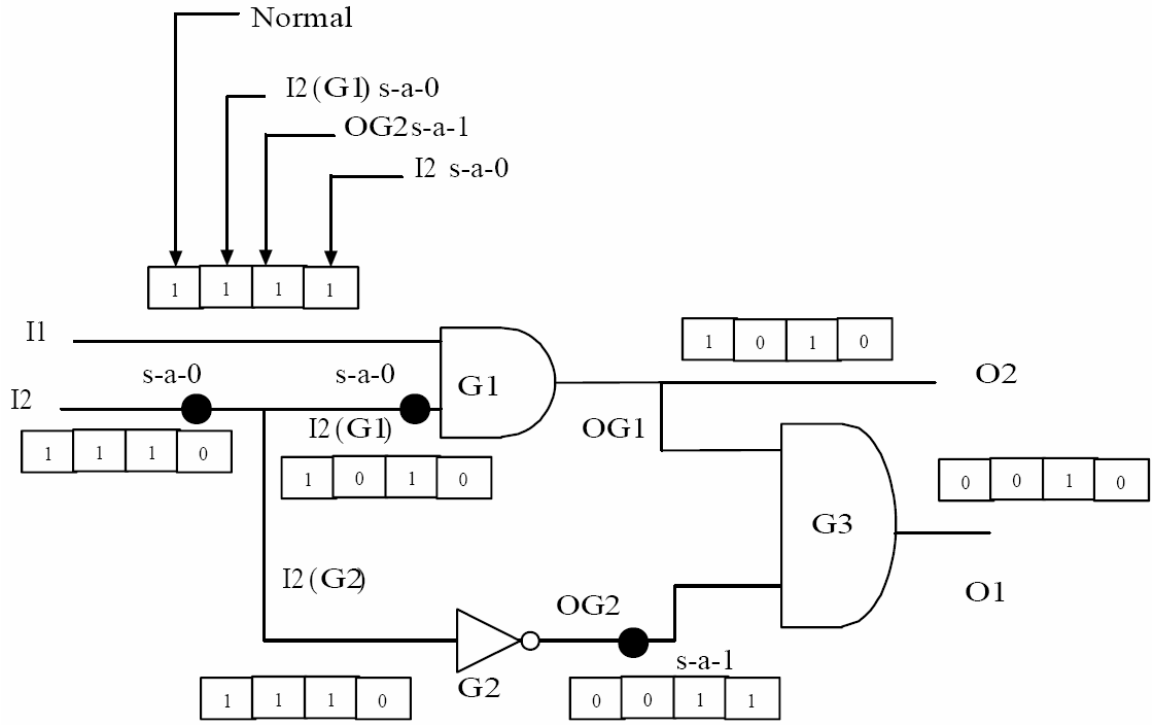


Figure 10. Parallel fault simulation example

It may be noted that each net of the circuit has a 4 word array, where the

- first bit corresponds to normal condition of the circuit,
- the second for s-a-0 fault at I2(G1),
- the third bit for s-a-1 fault at OG2,
- the fourth for s-a-0 at I2.

The first bit of the array at I1 is 1, implying that under normal condition I1=1 (i.e., input I1 is 1 by the random pattern). Similarly, input I2 is 1 by the random pattern. The second

bit of the array for I1 is 1 because under the s-a-0 fault at I2(G1), I1 has value of 1; similarly the third and fourth bits of the array for I1 are 1. In a similar way, the second and third bits of the array at I2 are 1. However, the fourth bit of the array at I2 is 0, because under the s-a-0 fault at I2, net I2 has value of 0.

The array at I2(G1) is a replica of the array at I2, but after changing the second bit to 0 as under s-a-0 fault at I2(G1), net I2(G1) has the value 0. The array at I2(G2) is a replica of the array at I2 as no fault is considered at I2(G2).

So, to fill the values of the array in a fanout branch when the values at the stem are known, there are two steps

1. *Copy the values of array of the stem to the arrays of the branches*
2. *If there is any s-a-0 (or s-a-1) fault is a branch, change the corresponding bit of the array to 0 (or 1).*

The array at OG1 is obtained by parallel logic AND operation on the words at the input lines I1 and I2(G1). The array at OG2 is obtained by parallel logic NOT operation on the word at the input line I2(G2) and making the third bit as 1 (as it corresponds to s-a-1 fault at OG2). *So, to fill the values of the array in the output of a gate when the values at the input are known, there are two steps*

1. *Obtain the values of array by parallel logic operation on the bits of the input words.*
2. *If there is s-a-0 (or s-a-1) fault at the output of the gate, change the corresponding bit of the array to 0 (or 1).*

In a similar way the whole example can be explained.

The array at O1 is 0010. It implies that on the input I1=1 and I2=1

- O1 is 0 under normal condition
- O1 is 0 under s-a-0 fault at I2(G1),
- O1 is 1 under s-a-1 fault at OG2,
- O1 is 0 under s-a-0 at I2.

It can be seen that only s-a-1 fault at OG2 causes a measurable difference at primary output O1 under normal and faulty condition for input pattern I1=1,I2=1. So pattern

I1=1,I2=1 can detect only s-a-1 fault at OG2 (at output O1) but cannot detect s-a-0 fault at I2(G1) and s-a-0 fault at I2.

The array at O2 is 1010. It implies that on the input I1=1 and I2=1

- O2 is 1 under normal condition
- O2 is 0 under s-a-0 fault at I2(G1),
- O2 is 1 under s-a-1 fault at OG2,
- O2 is 0 under s-a-0 fault at I2.

It can be seen that s-a-0 fault at I2(G1) and s-a-0 fault at I2 cause a measurable difference at primary output O2 under normal and faulty condition for input pattern I1=1,I2=1. So pattern I1=1,I2=1 at output O2 can detect s-a-0 fault at I2(G1) and s-a-0 fault at I2 but cannot detect s-a-1 fault at OG2. However, I1=1,I2=1 at output O1 detects s-a-1 fault at OG2. So all the three faults are detected by I1=1,I2=1.
Thus, under once scan of the circuit, information about three faults for a random pattern is discovered. It may be noted from Figure 7, that three scans of the circuit were required to find the same fact. So parallel fault simulation, speeds us the serial fault simulation scheme by *w-1* times.

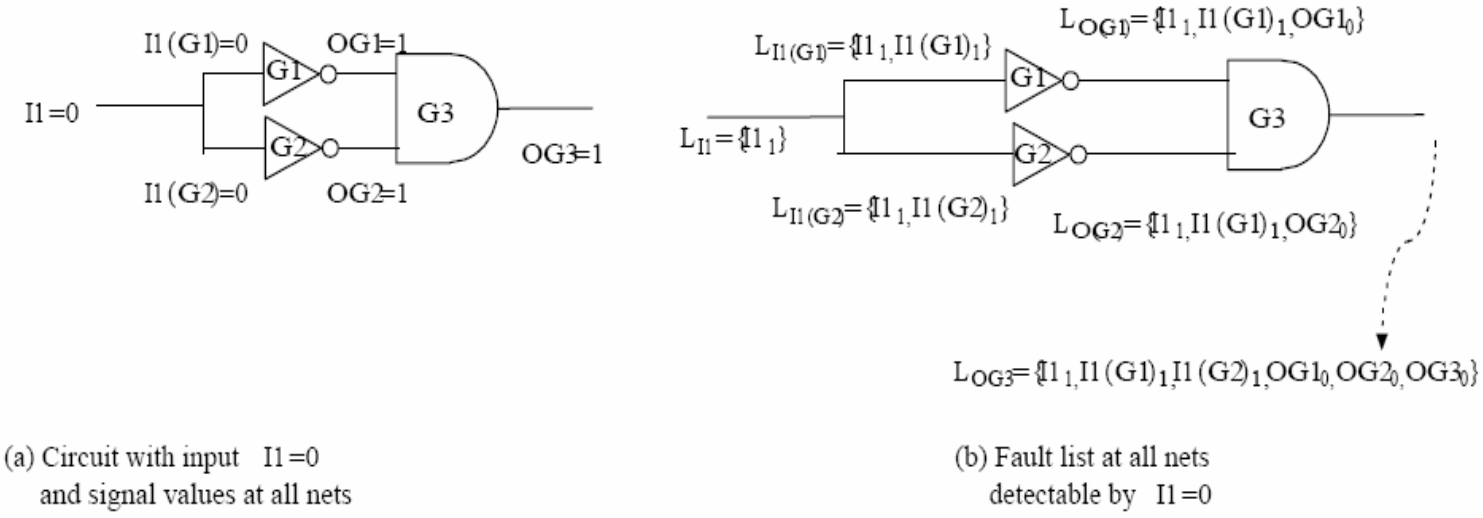
After an iteration of parallel fault simulation, next set of *w-1* faults are considered and the procedure repeated. After all the faults are considered (i.e., *total number of faults/(w-1)* iterations) the ones detected by the random pattern are dropped. Next another random pattern is taken and a new set of iterations are started.

So parallel fault simulation speeds up serial fault simulation *w-1* times, but for a random pattern more than one iterations are required. In the next section we will see a scheme where in one iteration, information about ALL faults for a random pattern can be generated.

3.3 Deductive Fault Simulation

Form the discussion in the last section it can be noted that parallel fault simulation can speed up the procedure only by a factor that is dependent on the bit width of the computer being used. In this section we will discuss about deductive fault simulation, a procedure which can determine in a single iteration, detectability/undetectability about all faults by a given random pattern. In the deductive method, first the fault-free circuit is simulated by a random pattern and all the nets are assigned the corresponding signal values. "Deductive", as the name suggests, all faults detectable at the nets are determined using the structure of the circuit and the signal values of the nets. Since the circuit structure remains the same for all faulty circuits, all deductions are carried out simultaneously. Thus, a deductive fault simulator processes all faults in a single pass of simulation augmented with the deductive procedures. Once detectability of all the faults for a random pattern is done, the same procedure is repeated for the next random pattern after eliminating the covered faults.

We will explain the procedure by a simple example circuit given in Figure 11.



(a) Circuit with input I1=0 and signal values at all nets

(b) Fault list at all nets detectable by I1=0

Figure 11. Example of deductive fault simulation with input I1=0

The circuit with input I1=0 and the signal values at all nets are given in Figure 11(a). Figure 11 (b) shows the fault list that can be detected at each net when I1=1. For a net I say, L_I denotes the fault list of I. Also, $L_I = \{I_0\}$ ($L_I = \{I_1\}$) would denote that s-a-0 (s-a-1) at net I is detectable at I (for the present value of signal at I).

Now we see stepwise how fault list can be determined for all the nets of the circuit (Figure 11) using deductive simulation.

- Step-1: $L_{I1} = \{I1_1\}$. s-a-1 at I1 can be detected at I1 as input pattern is I1=0.
- Step-2: $L_{I1(G1)} = \{I1_1, I1(G1)_1\}$. s-a-1 at I1 can be detected at fanout branch I1(GI). Also s-a-1 fault at the fanout branch I1(GI) can be detected at I1(GI) because the signal value at I1(GI)=0, for the given input. Similarly, $L_{I1(G2)} = \{I1_1, I1(G2)_1\}$. *Thus the following rule about fault deduction at fanout branches can be made*
    - *Fault list detected at fanout branch comprise (i) all faults at the fanout stem and (ii) s-a-1 if the signal value at the branch is 0, else s-a-0.*
- Step 3A: After the inverter G1 (OG1), it may be noted that $I1_1$ is detectable because if circuit is normal then OG1=1, and if I1 is s-a-1 then OG1=0. Similarly, $I1(G1)_1$ is also detectable. Now, as under normal condition OG1=1 for input I1=0, s-a-0 at OG1 is detectable at OG1. So, $L_{OG1} = \{I1_1, I1(G1)_1, OG1_0\}$. *Thus the following rule about fault deduction at an inverter can be made*
    - *Fault list detected at output of an inverter comprise (i) all faults at the input and (ii) s-a-1 of the signal value at the output is 0, else s-a-0.*
- Step 3B: Similar to the logic used in Step-3A, $L_{OG2} = \{I1_1, I1(G2)_1, OG2_0\}$.
- Step 4. For the AND gate G3, it may be noted that for input I1=0, both its inputs are 1. So, if OG1 is s-a-0, it can be detected and also if OG2 is s-a-0 it can be detected; OG3 is 1 under normal conditions and OG3=0 under these faults. It may be verified that all faults in the list L_{OG1} and L_{OG2} can be detected at the output of the AND gate. This is possible because under normal condition and input I1=0, both the inputs of the AND gate are 1. If one input of a 2-input AND gate is 1, then all faults detectable at the other input are also detectable at the output of the AND gate. However, if one input of a 2-input AND gate is 0, then no fault detectable at the other input remains so as the output of the AND gate is 0 with or

without faults. Also, s-a-0 fault at the output of the AND gate is detectable at OG3 because under normal condition OG3=1. So, L_{OG3}={ $I1_1$, $I1(G1)_1$, $I1(G2)_1$, $OG1_0$,$OG2_0$,$OG3_0$}. *Thus the following rule about fault deduction at a 2-input AND gate with both inputs as 1 can be made*

- *Fault list detected at output of the AND gate comprise (i) all faults at both the inputs and (ii) s-a-0 at the output of the AND gate.*

Next we will see the fault deductions at the various nets if I1=1. This situation is illustrated in Figure 12.

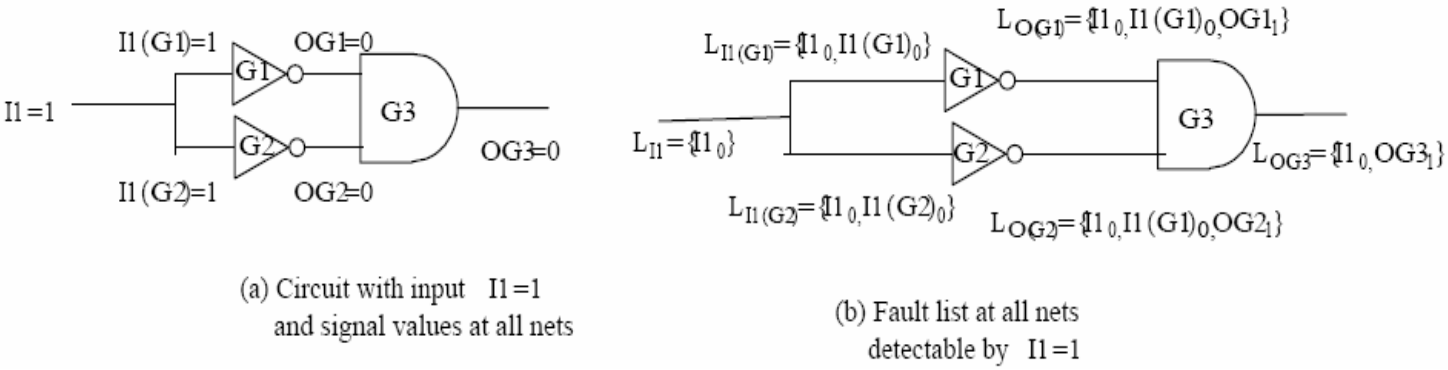


Figure 12. Example of deductive fault simulation with input I1=1

- Step-1: L_{I1}={ $I1_0$}. s-a-0 at I1 can be detected at I1 as input pattern is I1=1.
- Step-2: $L_{I1(G1)}$={ $I1_0$, $I1(G1)_0$}. s-a-0 at I1 can be detected at fanout branch I1(GI). Also s-a-0 fault at the fanout branch I1(GI) can be detected at I1(GI) because the signal value at I1(GI)=1, for the given input. Similarly, $L_{I1(G2)}$={ $I1_0$, $I1(G2)_0$}.
- Step 3A: After the inverter G1 (OG1), it may be noted that $I1_0$ is detectable because if circuit is normal then OG1=0, and if I1 is s-a-0 then OG1=1. Similarly, $I1(G1)_0$ is also detectable. Now, as under normal condition OG1=0 for input I1=1, s-a-1 at OG1 is detectable at OG1. So, L_{OG1}={ $I1_0$, $I1(G1)_0$,$OG1_1$}.
- Step 3B: Similar to the logic used in Step-3A, L_{OG2}={ $I1_0$, $I1(G2)_0$,$OG2_1$}.
- Step 4. For the AND gate G3, it may be noted that for input I1=1, both its inputs are 0. It may be verified that excepting I1 s-a-0, no other fault (in the lists L_{OG1} and L_{OG2}) can be detected. If I1 is s-a-0, then output of the AND gate is 1 and is 0 under normal condition. As both the inputs are of the AND gate are 0, any fault

ONLY at OG1, s-a-0 at I1(G1) say, cannot be detected because output of AND gate is 0 with or without the s-a-0 fault at I1(G1). Similarly, any fault ONLY at OG2, s-a-0 at I1(G2) say, cannot be detected. I1 s-a-0 can be detected as it affects both the inputs of the AND gate. Also, s-a-1 fault at the output of the AND gate is detectable at OG3 because under normal condition OG3=0. So, $L_{OG3}=\{ I1_{0,} OG3_1\}$. *Thus the following rule about fault deduction at a 2-input AND gate with both inputs as 0 can be made*

- *Fault list detected at output of the AND gate comprise (i) all faults COMMON at both the inputs and (ii) s-a-1 at the output of the AND gate.*

So, now the question remains, what are the rules for fault deduction at a 2-input AND gate with one input as 1 and the other 0. The rules are explained using another example in Figure 13.

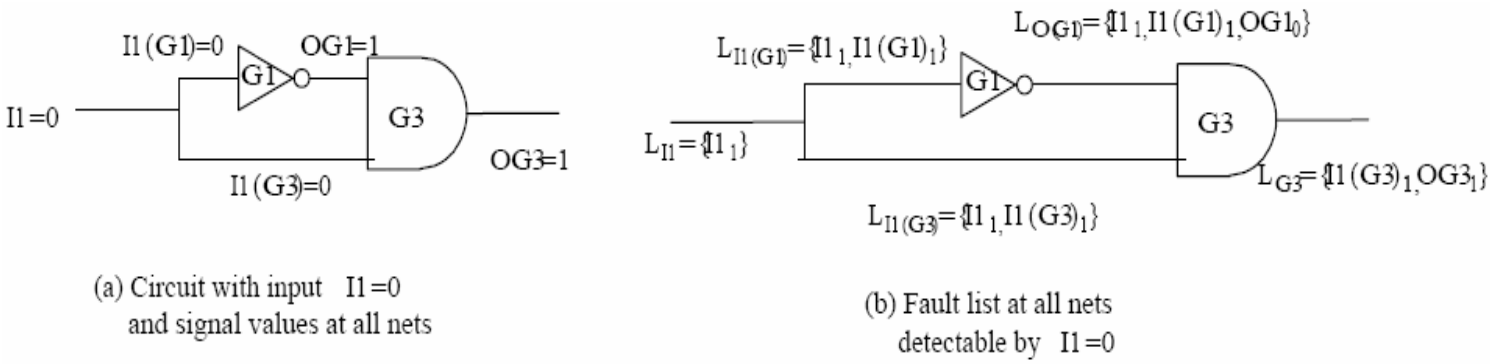


Figure 13. Example of deductive fault simulation with input I1=0 (Inputs of the AND gate are 1 and 0)

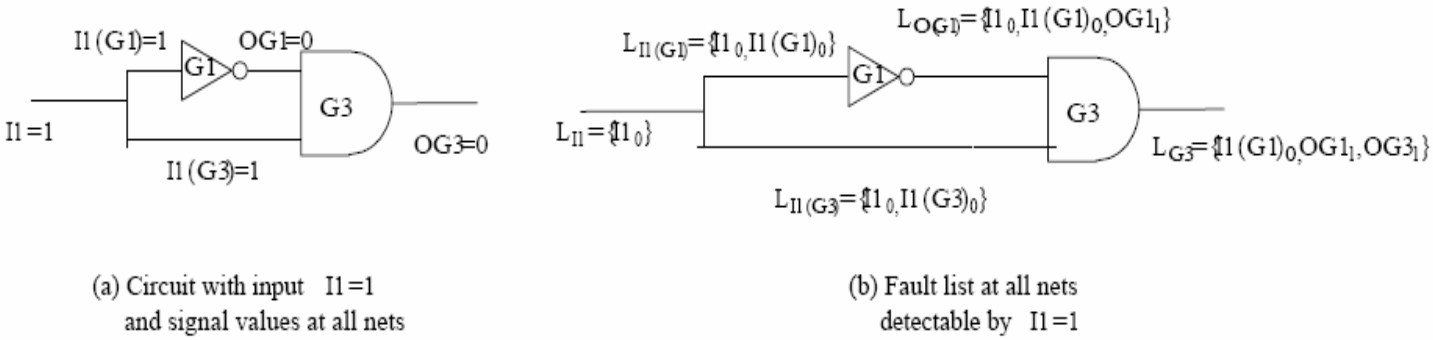
- Step-1: $L_{I1}=\{I1_1\}$
- Step-2: $L_{I1(G1)}=\{I1_{1,} I1(G1)_1\}$ and $L_{I1(G3)}=\{I1_{1,} I1(G3)_1\}$.
- Step 3: $L_{OG1}=\{I1_{1,} I1(G1)_{1,} OG1_0\}$.
- Step 4. For the AND gate G3, it may be noted that for input I1 set to 0, OG1=1 and I1(G3)=0. So, if OG1 is s-a-0, it cannot be detected as output of AND gate is

0 with and without s-a-0 fault at OG1. But if I1(G3) is s-a-1, it can be detected as output of AND gate is 1 with fault and 0 without fault. It may be verified that all faults at the list $L_{I1(G3)}$ can be detected. Further, none of the faults at the list L_{OG1} can be detected. So, all faults in $L_{I1(G3)}$ except the common fault ($I1_1$ for both L_{OG1} and $L_{I1(G3)}$) can be detected. Also, s-a-1 fault at the output of the AND gate is detectable at OG3 because under normal condition OG3=0. Thereby L_{OG3}={ $I1(G3)_1, OG3_1$}. *Thus, the following rule about fault deduction at a 2-input (In1 and In2) AND gate with In1=1 and In2=0 can be made*

- *Fault list detected at output of the AND gate comprise (i) all faults at In2 but not at In1 and (ii) s-a-1 at the output of the AND gate.*

Similarly, the following rule about fault deduction at a 2-input (In1 and In2) AND gate with In1=0 and In2=1 can be made

- *Fault list detected at output of the AND gate comprise (i) all faults at In1 but not at In2 and (ii) s-a-1 at the output of the AND gate. This is illustrated in Figure 14.*



(a) Circuit with input I1 =1 and signal values at all nets

(b) Fault list at all nets detectable by I1 =1

Figure 14. Example of deductive fault simulation with input I1=1 (Inputs of the AND gate are 0 and 1)

Based on the discussion of Example 11 through example 14, the following table (Table 5) can be constructed that enumerates the rules for all logic gates required for fault deduction.

Table 5. Fault deduction rules for logic gates

Gate Type	Inputs (In1 and In2)		Output (O)	Deductive Fault list at O
AND	0	0	0	$[L_{In1} \cap L_{In2}] \cup O_1$
	0	1	0	$[L_{In1} \cap \overline{L_{In2}}] \cup O_1$
	1	0	0	$[\overline{L_{In1}} \cap L_{In2}] \cup O_1$
	1	1	1	$[L_{In1} \cup L_{In2}] \cup O_0$
OR *(Dual of AND gate)*	0	0	0	$[L_{In1} \cup L_{In2}] \cup O_1$
	0	1	1	$[\overline{L_{In1}} \cap L_{In2}] \cup O_0$
	1	0	1	$[L_{In1} \cap \overline{L_{In2}}] \cup O_0$
	1	1	1	$[L_{In1} \cap L_{In2}] \cup O_0$
NOT	0	----	1	$[L_{In1}] \cup O_0$
	1	-----	0	$[L_{In1}] \cup O_1$
Fanout	0	----	0	$[L_{In1}] \cup O_1$
	1	------	1	$[L_{In1}] \cup O_0$

This table is for 2 input gates only, however, these rules can be directly extended for 3 or more input gates. For example, for a 3 input AND gate (In1, In2, In3), with In1=1, In2=1, In3=1, the rule for deduction is $[L_{In1} \cup L_{In2} \cup L_{In3}] \cup O_0$ while for In1=1, In2=1, In3=0, the rule for deduction is $[\overline{L_{In1}} \cap \overline{L_{In2}} \cap L_{In3}] \cup O_1$.

Now we will revisit our running example circuit of Figure 7 and perform deductive fault simulation with pattern I1=1,I2=1; this is illustrated in Figure 15. From the rules in Table 5, it may be computed easily that $L_{O2}=\{I1_0,I2_0,I2(G1)_0,OG1_0,O2_0\}$ and $L_{O1}=\{I2(G2)_0,OG2_0,O1_1\}$. By studying the fault list at the primary outputs (L_{O1} and L_{O2}), we find that deductive fault simulation discovered 8 faults detectable by pattern I1=1,I2=1; all other faults cannot be detected. These 8 faults are dropped and the process repeated for new pattern.

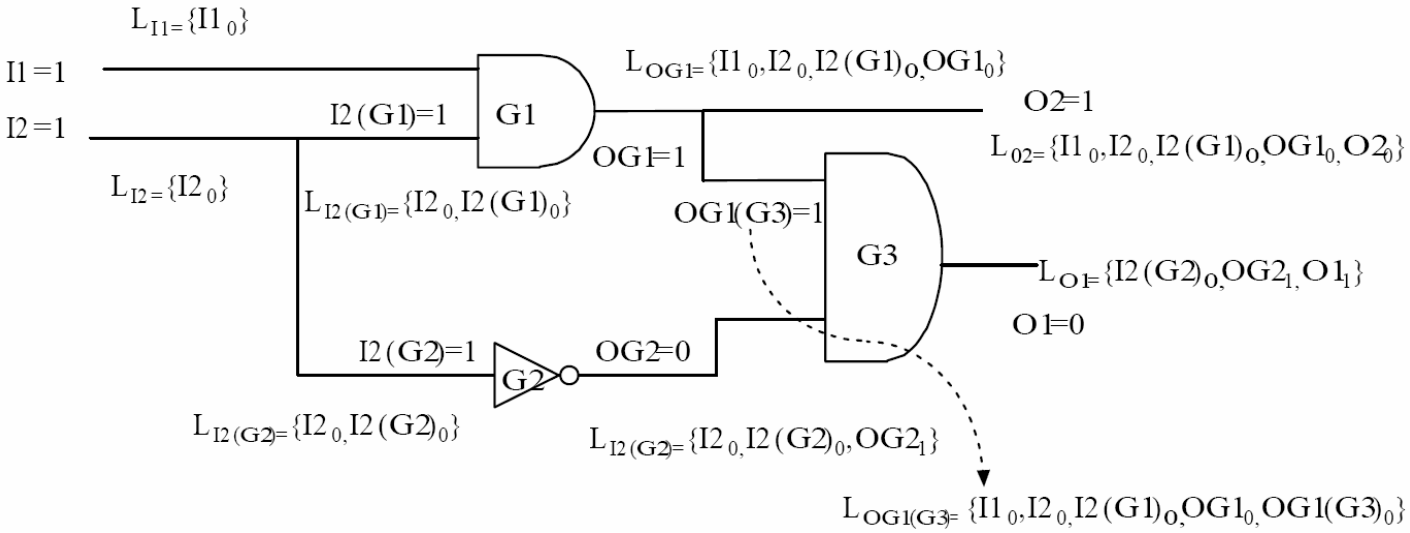


Figure 15. Deductive fault simulation with input I1=1 and I2=1

3.4 Concurrent Fault Simulation

From the discussion in the in the last section we found that detective fault simulation can determine all the faults in one iteration detectable by a random pattern. However, when a new random pattern is fed as input the whole process needs to be redone. "Concurrent Fault Simulation" is a technique similar to deductive fault simulation, however, retains information when moving from one random pattern to another. In other words, in concurrent fault simulation when a new random pattern is fed it needs to compute only that information which got changed by the new pattern. So, concurrent fault simulation gets motivation from the advantages achieved by event driven simulation compared to compiled code simulation.

In concurrent fault simulation to each gate is associated a number of gates "affected by some fault in the circuit". An affected gate is one whose at least one input or output is different from the ones in the original (normal gate). An example of such affected gates being added to an AND gate is given in Figure 16. In the example the input to the AND gate is I1=1 and I2=1. Now due to three faults, namely (i) s-a-0 at I1, denoted as $I1_0$ (ii) s-a-0 at I2, denoted as $I2_0$ and (iii) s-a-0 at OG1, denoted as $OG1_0$, some signals in the input or output of the AND gate differs compared to normal condition. So three AND gates "affected" by fault is attached to the AND gate. For example, in Figure 16 due to s-a-0 fault at I1, signal I1 at AND gate is 0 while it is 1 at normal condition; first gate of the "affected gate" list corresponds to this case. Also, the output of the AND gate is 0 under fault (versus 1 at normal condition). This fact is also captured by the first affected gate. Similarity all the other two affected gates can be explained.

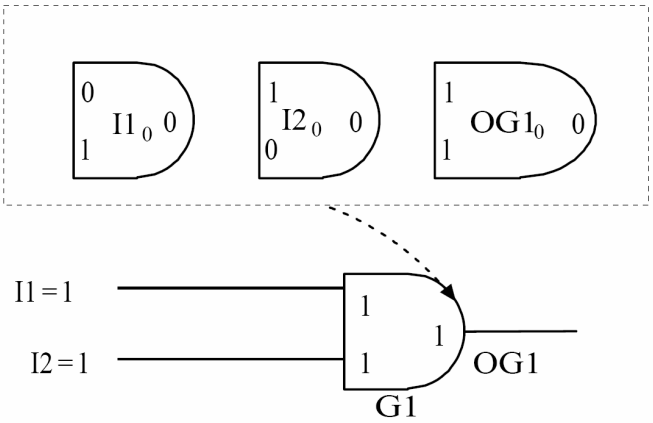


Figure 16. Fault affected gates being added to an AND gate (Concurrent Simulation)

Given a circuit, level wise affected gates corresponding to all normal gates are created. Now, affected gates (in the list of normal gates) that drive some primary output are considered. Among those affected gates, the ones whose output signal value differs from that of the normal gate, correspond to faults being detected by the random pattern given as input. This is explained by an example given in Figure 17.

In Figure 17, AND gate G1 can be affected by four faults, namely (i) s-a-0 at I1, (ii) s-a-0 I2, (iii) s-a-0 at I2(G2) and (iv) s-a-0 at OG1. So four affected gates with signals corresponding to the faults are shown in the list of G1. As G2 does not drive any primary output, so fault detectability is not computed for the list at G2. Similarly, NOT gate G2 can be affected by three faults, namely (i) s-a-0 at I2, (ii) s-a-0 I2(G2), (iii) s-a-1 at OG2. So three affected gates with signals corresponding to the faults are shown in the list of G2. As G2 does not drive any primary output, so fault detectability is not computed for the list at G2.

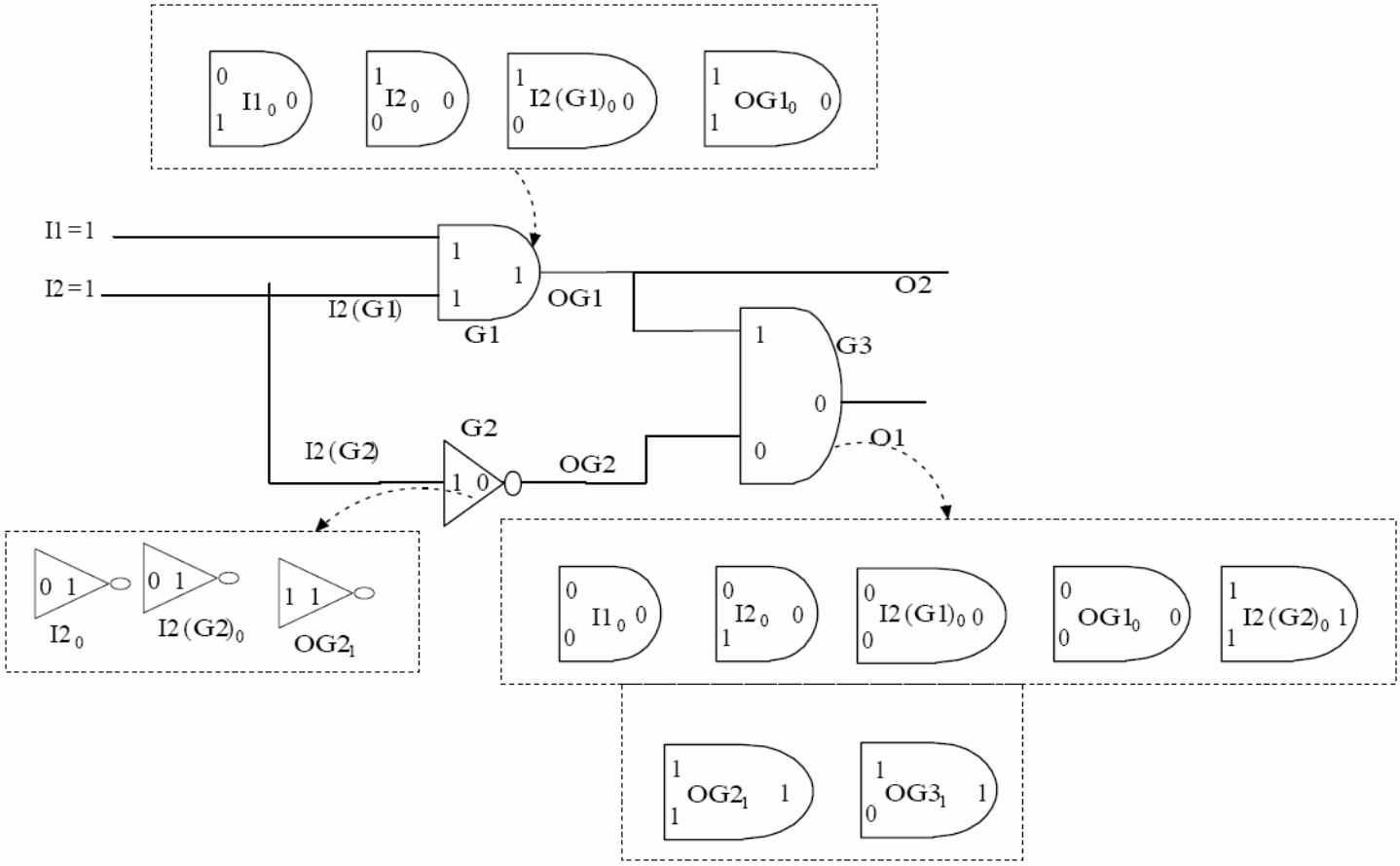


Figure 17. Example of concurrent fault simulation

Following that list of affected gates at G3 is computed using the list of the gates driving it (namely, G1 and G2). For example, under s-a-0 fault at I1, OG1=0, OG2=0 and OG3=0 (while under normal condition OG1=1, OG2=0 and OG3=0); this is captured by affected gate marked $I1_0$ in the list of for G3. Similarly, for the other five faults (i) s-a-0 at I2, (ii) s-a-0 at I2(G1), (iii) s-a-0 OG1, (iv)s-a-0 at I2(G2), (v) s-a-1 at OG2 , there are five gates in the list of G3. Also, another gate for s-a-1 fault at OG3 is added to the list making the number of affected gates at G3 to seven.
G3 drives driving primary output O1 and in the entire list of affected gates, output signal is 1 for (i) s-a-0 at I2(G2), (ii) s-a-1 at OG2 and (iii) s-a-1 at OG3, while in the normal gate output is 0. So input I1=1,I2=1 detects these three faults at O1.

It may be noted that only three gates correspond to faults being detected at OG3 (or primary output O1). So what is requirement of seven gates in the affected list? It may appear that deductive fault simulation, which keeps information about only these three faults (see Figure15) is better choice than concurrent fault simulation. The answer comes if we consider then next random pattern as I1=0, I2=1. In case of deductive fault simulation we need to repeat all the steps, while in case of concurrent simulation we will re-compute only the information that changes as a result of changes in signals triggered by I1 (from 1 to 0). Now we will see the steps involved in the example of Figure 17 when input changes from I1=1, I2=1 to I1=0, I2=1.

Figure 18 illustrates the case when input I1 changes from 1 to 0. It may be observed that due to change in I1, signal in both the normal gates and the ones in the affected list change. For example, in gate G1, first input changes from 1 to 0 and also OG1 changes from 1 to 0. Also, the first input in affected gates $I2_0$ and $OG1_0$ change from 1 to 0. After these changes signal values in the affected gates $I1_0$ and $OG1_0$ become same as that of the normal gate G2. So affected gates $I1_0$ and $OG1_0$ are deleted from the list. Similarly, signals in affected gates $I1_0$, $I2(G1)_0$ and $OG1_0$ become same as that of normal gate G3; such gates are also deleted. It may be noted that some (affected) gates remain which has signal difference compared to normal gate even when input changes to I1=0,I2=1; for example, $I2_0$ remains in the list of OG1. *Information about these (remaining) gates are*

retained from the previous input I1=1,I2=1; retaining this information over change in input patters is the main advantage of concurrent fault simulation.

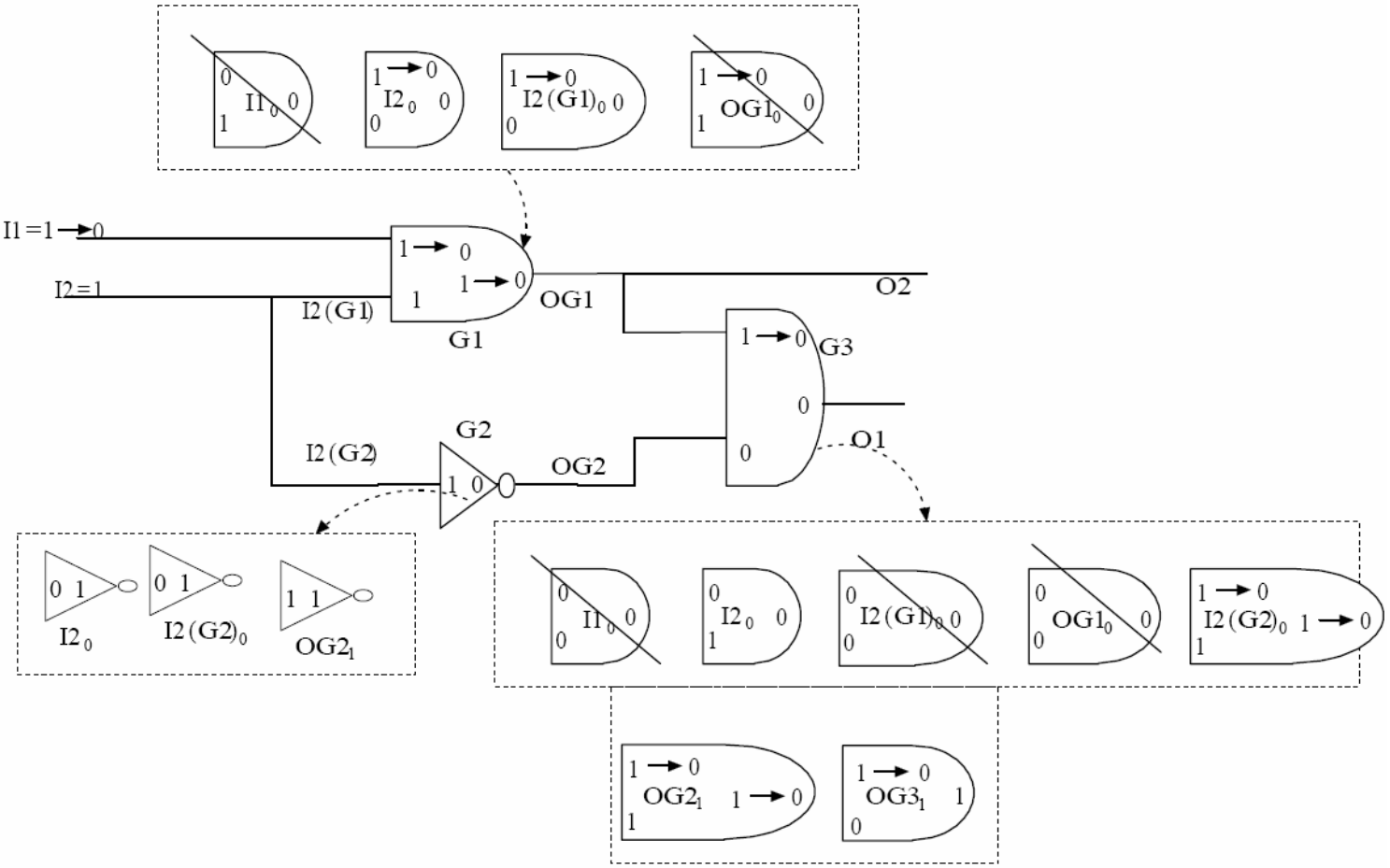


Figure 18. Gates deleted if input changes from I1=1, I2=1 to I1=0, I2=1 (example of the circuit in Figure 17)

The story is not yet over. As some gates get eliminated from the affective list when input pattern changes, also some new gates get added to the list. Figure 19 shows the remaining affected gates and the ones added after change in input. For example, for s-a-1 fault at I1, the gate $I1_1$ gets added to the list of G1, because first input and output of $I1_1$ is different compared to G1. Similarly, another gate for s-a-1 fault at OG1 gets added to the list of G1. So affected gate list of G1 has four gates. G1 drives primary output OG1 and outputs of the affected gates $I1_1$ and $OG1_1$ are different compared to G1. So s-a-1 in I1 and s-a-1 at OG1 are detected by input 11=0,I2=1. Similarly, in case of primary output O1, s-a-1 fault in OG3 is detected by 11=0,I2=1.

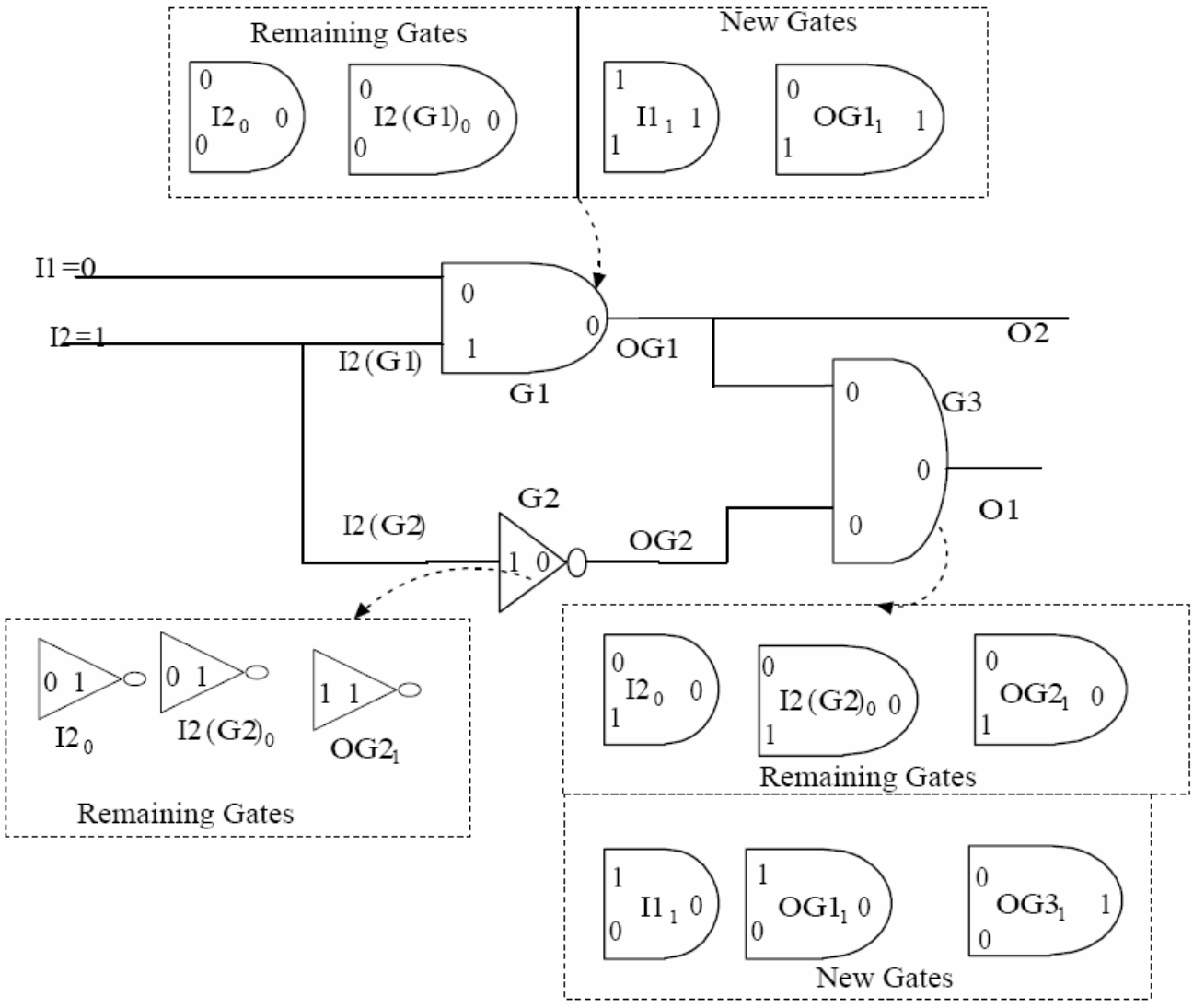


Figure 19. Remaining affected gates and the ones added after change in input.

To conclude, fault simulation algorithms help to determine patters that can test a subset of faults in a circuit. Broadly speaking, after about 90% of faults being detected by random patters and fault simulation, we need to go for ATPG by sensitization–propagation -justification approach. Now, if there was a scheme that could tell which 90% of faults are easy to test (by random patterns) and which are difficult to test, then fault simulation algorithms could be more focused. In other words, fault simulation algorithms would stop when most of the easy faults were covered. In the next lecture we would see an approximate and fast algorithm, which predicts which faults are easy to test and which are difficult.

References

[1]. S. H. Chisholm and L. W. Nagel, "Efficient Computer Simulation of Distortion in Electronic Circuits," IEEE Trans. on Circuit Theory, vol. CT-20, no. 6, pp. 742–745, Nov. 1973.

[2]. W. J. McCalla, Fundamentals of Computer-Aided Circuit Simulation. Boston: Kluwer Academic Publishers, 1988.

[3]. B. R. Chawla, H. K. Gummel, and P. Kozak, "MOTIS – An MOS Timing Simulator," IEEE Trans. on Circuits and Systems, vol. CAS-22, no. 12, pp. 901–910, Dec. 1975.

[4]. D. R. Coelho, "The VHDL Handbook", Boston: Kluwer Academic Publishers, 1989.

Question and Answers:

1. What is the main advantage of compiled code simulator versus event driven simulator?

Answer: In case of compiled code simulator the whole circuit (i.e., all gates) needs to be simulated when input pattern changes. In case of event driven simulation, only those gates are required to be simulated whose inputs change because of change in the input. So, event driven simulation is more time efficient compared to compiled code simulation.

2. Which fault simulation algorithm is most dependent on architecture of the computer simulating it?

Answer: Concurrent fault simulator is mostly dependent on the architecture of the computer simulating it because speed (i.e., number of faults simulated per iteration) depends on the bit width of the word of the computer. If the bit width is *w,* then *w-1* faults can be simulated in an iteration.

3 When the procedure for test pattern generation by fault simulation is stopped and ATPG by sensitization–propagation –justification approach is taken?

Answer: Under two cases, test pattern generation by fault simulation is stopped.

(i) A high percentage of faults are detected by fault simulation; it implies that most of the easy to test faults are covered and the ones remaining are difficult to test faults.

(ii) For a significant number of random patterns at a stretch, new faults covered are nominal; it implies that in the circuit a substantial percentage of faults are difficult to test and one should resort to sensitization–propagation –justification approach.